# Reporte Técnico de Actividades Práctico-Experimentales Nro. 005

## 1. Datos de Identificación del Estudiante y la Práctica:

| | |
|---|---|
| **Nombre del estudiante(s)** | Matías Morocho |
| **Asignatura** | Teoría de la Programación |
| **Ciclo** | 1 “A” |
| **Unidad** | 1 |
| **Resultado de aprendizaje de la unidad** | Identifica los conceptos fundamentales de la teoría de la programación, bajo los principios de solidaridad, transparencia, responsabilidad y honestidad. |
| **Práctica Nro.** | 005 |
| **Título de la Práctica** | Individual |
| **Nombre del Docente** | José Oswaldo Guamán Quinche |
| **Fecha** | Jueves 29 de octubre del 2026 al Viernes 20 de octubre del 2026 |
| **Horario** | Jueves, 12h30 – 13h30<br>Viernes, 09h30 - 11h30 |
| **Lugar** | Aula física asignada al paralelo. |
| **Tiempo planificado en el Sílabo** | 3 horas |

## 2. Objetivo(s) de la Práctica:

● Desarrollar la capacidad de transformar un problema en una solución computacional.
● Aplicar estructuras secuenciales en el diseño del algoritmo en un lenguaje de programación.
● Validar la lógica del algoritmo mediante pruebas de escritorio.

## 3. Materiales, Reactivos, Equipos y Herramientas:

● IDE de programación: Visual Studio Code u otro entorno compatible.
● Lenguaje de programación: Según los contenidos de la unidad.
● Computador personal con sistema operativo Windows, Linux o macOS.
● Material de apoyo en el Entorno Virtual de Aprendizaje (EVA).
● Editores de texto (Word, Google Docs u otros) para la elaboración del informe técnico en formato PDF.
● Conexión a internet estable para acceder a recursos digitales y software en línea.

## 4. Procedimiento / Metodología Ejecutada:

Para esta práctica la metodología que se usó fue analizar el problema planteado por el docente, identificando los datos de entrada, proceso y salida de el mismo, para posteriormente realizar el pseudocódigo y diagrama de flujo, usando draw.io, luego se verificó la resolución del problema con pruebas de escritorio.

Finalmente se ejecutó el algoritmo en VS Code comprobando su funcionalidad a través de la comparación de los resultados del código con las pruebas de escritorio.

## 5. Resultados:

**Análisis del problema:**

- Datos de entrada:

Primer certamen (C1), segundo certamen (C2), nota de laboratorio (NL).

- Proceso:

C3= ((3(60 – (NL * 0.3))) / 0.7) - C1 - C2

NC= (C1 + C2 + C3) / 3

NF= (NC * 0.7) + (NL * 0.3)

- Datos de salida:

Presentar la nota que necesita un estudiante en el tercer certamen (C3) para aprobar.

**Pseudocódigo:**

Inicio

C1= 0.0;
C2= 0.0;
C3= 0.0;
NL= 0.0;
NC= 0.0;
NF= 0.0;

Leer C1;
Leer C2;
Leer NL;

C3= ((3*(60 – (NL * 0.3))) / 0.7) - C1 - C2;
NC= (C1 + C2 + C3) / 3;
NF= (NC * 0.7) + (NL * 0.3);

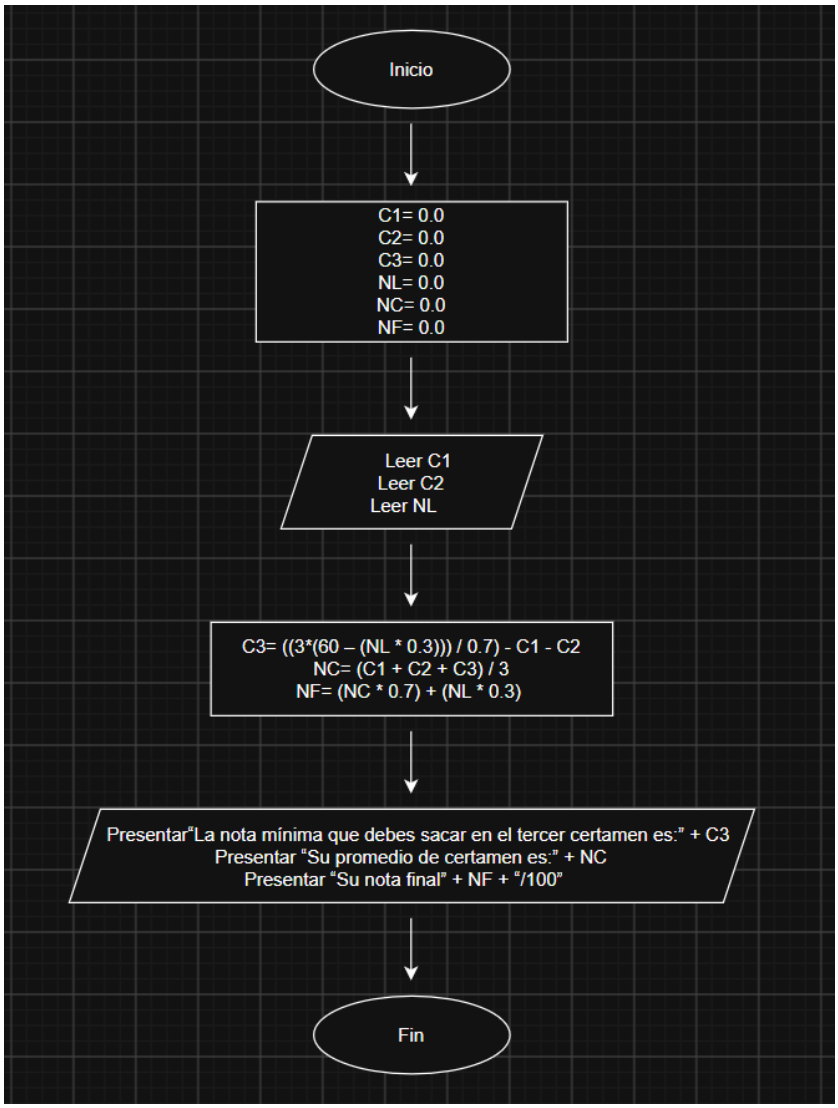
Presentar “La nota mínima que debes sacar en el tercer certamen es:” + C3;
Presentar “Su promedio de certamen es:” + NC;
Presentar “Su nota final” + NF + “/100”;

Fin

**Pruebas de escritorio:**

| C1 | C2 | NL | Nota mínima en C3 | NC | NF |
|---|---|---|---|---|---|
| 60 | 70 | 55 | 56.43 | 62.14 | 60 |

**Diagrama de Flujo:**



**Código:**

aprobar.c

```c
#include <stdio.h>

void main (){
    //Declaración de variables
    float C1 = 0.0;
    float C2 = 0.0;
    float C3 = 0.0;
    float NL= 0.0;
    float NC= 0.0;
    float NF= 0.0;

    //Solisitar datos al usuario
    printf("Ingrese la nota del primer certamen:\n");
    scanf("%f", &C1);

    printf("Ingrese la nota del segundo certamen:\n");
    scanf("%f", &C2);

    printf("Ingrese la nota de laboratorio:\n");
    scanf("%f", &NL);

    //Cálculos para la resolución del problema
    C3= ((3*(60-(NL * 0.3))) / 0.7) - C1 - C2; //Se despeja C3 usando las 2 formulas planteadas
    NC= (C1 + C2 + C3) / 3;
    NF= (NC * 0.7) + (NL * 0.3); //Debe dar como resultado 60/100, nota mínima para aprobar la materia

    //Presentar los resultados

    printf("La nota minima que debes sacar en el tercer certamen es: %.2f\n", C3); /*/Si sale negativo,
    aprueba con las notas actuales, y si sale mas de 100 reprobó/*/

    printf("Su promedio de certamenes es: %.2f\n", NC);
    printf("Su nota final es:%.2f /100\n", NF);
}
```

```
Problems  Output  Debug Console  Terminal  Ports

C:\Users\Usuario\Documents\PracticasC\deberes>gcc aprobar.c -o aprobar

C:\Users\Usuario\Documents\PracticasC\deberes>aprobar
Ingrese la nota del primer certamen:
60
Ingrese la nota del segundo certamen:
70
Ingrese la nota de laboratorio:
55
La nota minima que debes sacar en el tercer certamen es: 56.43
Su promedio de certamenes es: 62.14
Su nota final es:60.00 /100
```

## 6. Preguntas de Control:

- **¿Qué es un programa con estructuras secuenciales?**

Un programa de estructuras secuenciales es aquel que se ejecuta en un orden lineal, secuencial y una sola vez, con una estructura de datos de entrada, proceso y salida, sin el uso de repetidores o condicionales.

- **¿Cómo se traslada un algoritmo en pseudocódigo o diagrama de flujo a un lenguaje de programación?**

Para trasladar un algoritmo en pseudocódigo o diagrama de flujo, se debe realizar un proceso llamado “codificación” el cual consiste en transcribir lo escrito en el pseudocódigo a un lenguaje de programación.

- **¿Por qué es importante validar el programa mediante casos de prueba?**

Es importante ya que nos permite identificar si existe algún error en nuestro pseudocódigo y poder corregirlo, antes de pasar a la codificación,

## 7. Conclusiones:

- Se logro solucionar el problema planteado a través de la realización de un pseudocódigo, pruebas de escritorio y codificación, verificando la funcionalidad del algoritmo.
- Se uso estrictamente estructuras secuenciales para resolver el algoritmo, sin el uso de “if” “else” ni repetidores.
- Se verificó que el pseudocódigo sea correcto mediante pruebas de escritorio, para posteriormente compararlas con los resultados arrojados x la codificación del algoritmo.

## 8. Recomendaciones:

- Se recomienda realizar primero la prueba de escritorio para evitar futuros errores a la hora de codificar.
- Se recomiendo ir documentando el algoritmo para así evitar confusiones.

## 9. Bibliografía / Referencias:

No se usó citas bibliográficas para las preguntas de control.

## 10. Anexos:

https://github.com/Matias08xm/teoria_de_la_programacion-/blob/303654a508654a155887da9d6a9925905e545ce4/unidad_1/codigos_deberes/APE5.c